

Assessing the Memory Wall in Complex Codes

[‡]Galen M. Shipman, Jered Dominguez-Trujillo, Kevin Sheridan, Sriram Swaminarayan

Computer, Computational, and Statistical Sciences

Los Alamos National Laboratory

Los Alamos NM, USA

gshipman@lanl.gov, jereddt@lanl.gov, kss@lanl.gov, sriram@lanl.gov

Abstract—Many of Los Alamos National Laboratory’s (LANL) High Performance Computing (HPC) codes are heavily memory bandwidth bound. These codes often exhibit high levels of sparse memory access which differ significantly from industry standard benchmarks such as STREAM and GUPS. In this paper we present an analysis of some of our most important code-bases and their memory access patterns. From this analysis we generate representative micro-benchmarks that preserve the memory access characteristics of our codes using two approaches, one based on statistical sampling of relative memory offsets in a sliding time window at the function level and another at the loop level. The function level approach is used to assess the impact of advanced memory technologies such as LPDDR5 and HBM3 using the gem5 [1] simulator. Our simulation results show significant improvements for sparse memory access workloads using HBM3 relative to LPDDR5 and better scaling on a per core basis. Assessment of two different CPU architectures show that significantly higher peak memory bandwidth results in high bandwidth on sparse workloads. These two assessments demonstrate the benefits of this workload characterization technique in memory system design and evaluation.

Index Terms—hardware architecture, memory technology, workload analysis

I. INTRODUCTION

Historically, the floating point operations per second (FLOPs) of a given architecture served as a reasonable proxy to total application performance in high performance computing (HPC). As FLOP instructions per cycle (IPC) has increased by orders of magnitude over that of memory IPC and as our HPC codes have grown increasingly complex with highly sparse data structures, this is no longer the case. In most respects, dense FLOPs are a solved problem and memory bandwidth for dense and, especially, sparse workloads is the primary performance challenge in HPC architecture. As illustrated in Figure 1 the memory bandwidth (BW) Byte-per-flop has reduced by 3x in just the past decade alone. Many of the benchmarks relied upon for making large investments in HPC do not adequately represent the primary bottlenecks of our HPC codes. The minor uptick in memory bandwidth/flop in 2020 is a result of the Fugako AF64X [2] based system. More recent systems such as Frontier [3] have continued the downward trend of this metric. To address this continued and

This work was supported by the U.S. Department of Energy through the Los Alamos National Laboratory (LANL). LANL is operated by Triad National Security, LLC, for the National Nuclear Security Administration of U.S. Department of Energy (Contract No. 89233218CNA000001). LA-UR-22-28607.

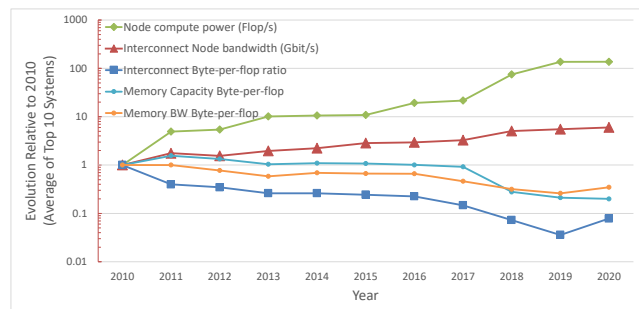


Fig. 1. System balance trends of the top 10 systems from the Top 500 list from 2010 through 2020

growing imbalance with deep co-design, the HPC community must improve in our ability to quantify the impact of the memory wall in our workloads.

II. RELATED WORK

Traditional HPC benchmarks such as LINPACK [4] are woefully insufficient, even as a simple proxy, for total system performance in HPC. The HPCG [5] benchmark is closer to the observed arithmetic intensity of some HPC applications, with approximately 4 memory load operations for each floating point operation. The STREAM [6] benchmark has focused on streaming contiguous memory and floating point operations (copy, sum, scale, triad). The Xstreams [7] benchmark, introduced an extension of the STREAM benchmark, adding an indirection array to these operations where copy becomes $A[i] = B[C[i]]$, scale becomes $A[i] = s * B[C[i]]$, and so on. This was done to better represent the indirection that is common in many scientific applications. More recently, the Spatter [8] benchmark (and associated workload capture tools) was developed to represent gather/scatter operations that are commonly used across a set of HPC applications. This benchmark provides a framework to replay gather/scatter patterns that are captured via a custom QEMU plugin and subsequent analysis. The Hopscotch benchmark [9] builds upon prior memory benchmarks, Spatter in particular, adding the ability to tune a working set size within a kernel and the ability to have mixed read/write workloads. An excellent overview of memory benchmarks is provided in Table 1 of the Hopscotch paper [9].

Both the STREAM and Spatter benchmarks are workloads that are well suited to drive architectural optimization at the hardware level due to their relative simplicity (small number of instructions). These workloads are tractable for hardware level simulation at a fairly detailed level in tools such as gem5/Ruby [1]. While STREAM provides an application agnostic workload, Spatter captures important patterns within a given application. This ability to more closely capture real-world application workloads in a compact form suitable for hardware level simulation is important to drive deeper hardware/software co-design.

While spatter and hopscotch can represent some types of scatter/gather patterns, they both lack the ability to represent more complex sparse memory operations from HPC applications.

III. WORKLOAD CHARACTERIZATION

To enable deeper co-design in memory technologies, an analysis of important production HPC applications developed at LANL was conducted. These codes are routinely run on some of the largest scale systems in the world such as Trinity and Sierra [10]. The analysis focuses on single node performance bottlenecks as prior work has eliminated the most significant network communication and storage system overheads, particularly for large-scale memory capacity limited problems (100's of terabytes of system memory in a single simulation) which is the focus of much of our work. The analysis presented here includes the Flag [11] arbitrary Lagrange-Eulerian hydrodynamics code that supports a wide-variety of unstructured mesh topologies, the xRAGE [12] Eulerian hydrodynamics code that supports cell-based adaptive mesh refinement, the PartiSN [13] discrete ordinate neutron transport code that supports a cartesian mesh topology, and the Jayenne [14] implicit Monte Carlo radiation transport code that supports a cartesian mesh topology. Two other codes are also analyzed here. All simulations run with these codes in this analysis are representative of commonly used code configurations.

Our workload characterization uses Intel's VTune to capture L1, L2, and L3 cache statistics, Memory bandwidth and latency statistics, floating point and vectorized floating point counters, and other top down [15] analysis information of the most computationally and data intensive components of our codes. Memory and cache statistics were captured using VTune's memory access and memory consumption collectors. Floating point ops and vector ops were captured using performance snapshot collectors. Frontend and backend μ ops were captured using μ arch collectors. Analysis of these collections was then performed to identify architectural bottlenecks on Intel Ice Lake class CPUs. The results of this analysis are illustrated in the heatmap in Figure 2 where green represents no architectural bottleneck while red represents a significant architectural bottleneck for the given application. For example, green in L1, L2 or L3 indicates there is no bottleneck on these caches, yellow indicates a moderate bottleneck on the caches, orange indicates a moderate to high bottleneck, while

red indicates a significant bottleneck. Green coloring of the Floating Point "DP FLOPS" cell indicates no bottleneck, while yellow is moderate, orange moderate to high, and red indicates a significant architecture bottleneck. An orange or red coloring of "DP FLOPS" generally indicates a high percentage of floating point operations within the instruction stream of the executing code. Floating point Vectorization again shows architecture bottlenecks, with green indicating little bottleneck on the SIMD / Vector units with yellow indicating a moderate bottleneck on floating point resources. A code which is moderately to significantly bottlenecked on vectorization is generally indicative of well-vectorized code and moderately high percentage of double precision floating point operations within the execution stream. With the exception of Flag, all of these codes show one or more bottlenecks resulting from the memory subsystem. Only the PartiSN code shows any significant bottleneck on floating point (DP FLOPs) performance with 26% of backend operations retired being DP FP operations and the majority (90.4%) of those DP FP ops being vectorized. All other codes show very small percentages of FP ops (14.3% to 2.5%) and vectorization of those FP ops between (19.2% and 0.2%). The final column simply shows the percentage of non-FP operations retired and is colored blue to differentiate this from the other values as this column does not map to a specific architecture bottleneck but is provided to reinforce the observation of the relatively small number of operations that are floating point within these code executions. Network communication, while an important component of many applications is not captured in this analysis.

Following this initial analysis, a deeper analysis of the xRAGE code focusing specifically on memory operations within the most memory intensive areas was conducted. Program instrumentation using Intel's PIN tool with customization was used to capture specific functions and memory operations within them. These multi-gigabyte logs were then parsed and loaded into an SQL database and processed Python. Initial analysis showed many of the memory operations are sparse within the captured functions. This result was intuitive as many of our codes use sparse physical and state space representations to reduce the required memory footprint for large 3D simulations to something tractable (hundreds of terabytes) on the largest HPC systems of today. To better understand the level of sparsity in this and other codes, a metric was developed, *windowed memory distance*, that captures the relative distance of memory accesses in a predefined window (temporal) of instructions. For a given memory read operation, $Read_K$, the subsequent N memory read operations $Read_{K+1:N}$ are tracked and the distance (in address space) is calculated for each. These values $Read_{K+1} - Read_K$, $Read_{K+2} - Read_K$, ..., $Read_N - Read_K$ are then used for the histogram of windowed memory distance. A positive value in the histogram indicates subsequent memory operations that are in higher address ranges than the current one and negative values indicate that the subsequent memory operations are in a lower address range. Heap operations are treated distinctly from stack operations and read operations are treated distinctly

	Memory subsystem						Floating Point		
	L1	L2	L3	DRAM	DRAM BW	Mem Latency	DP FLOPs	Vectorization	Non-FP
Flag 3D Ale							2.50%	7.10%	97.50%
PartiSN 42 groups							26.20%	90.40%	73.80%
Jayenne DDMC Hohlräum							14.30%	0.20%	85.70%
xRAGE Shaped Charge							6.50%	14.00%	93.50%
Application 1							7.80%	19.20%	92.20%
Application 2							8.10%	17.60%	91.90%

Fig. 2. Hardware bottlenecks in HPC codes at LANL

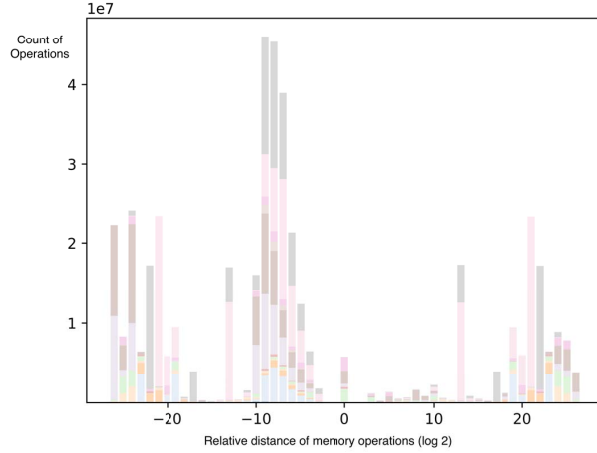


Fig. 3. Windowed memory distance metric with different colors representing individual routines/functions of the cell-based AMR code xRAGE.

from write operations. As we are primarily concerned with patterns of floating point memory operations, only the floating point load/store (scalar and vector) are considered in this analysis. The intent of this approach is to provide a way of visualizing the temporal and spatial locality of floating point memory operations within a running code. An example of this metric captured on read operations in the xRAGE code is illustrated in Figure 3. The majority of read operations are within 1024 bytes of one another which again matches our intuition. Loops within xRAGE are generally either iterating through leafs in a quad-tree (2D) or oct-tree (3D) which are ordered but non-contiguous and variable stride, or they are iterating through an auxiliary data structure that represents the faces of the cell to access neighbor cells which again are ordered but are non-contiguous and variable stride.

These non-contiguous, non-uniform access patterns that are based on real application workloads are not well-represented in current micro-benchmarks. Given their prevalence in a variety of data structures used in HPC this motivated two approaches to better represent these workloads which are discussed below.

IV. REPRESENTING SPARSE WORKLOADS

Our first approach to representing sparsity in micro-benchmarks focused on extensions to the STREAM bench-

mark. The modifications to STREAM were straight forward. First, an indirection array was introduced and incorporated into the Copy, Sum, Scale, and Triad loops. So the following STREAM Triad loop:

```
#pragma omp parallel for
for(j=0; j<STREAM_ARRAY_SIZE; j++)
a[j] = b[j]+scalar*c[j]
```

Is modified to include indirect lookups for both the read and write operations with an “r” and “w” array respectively.

```
#pragma omp parallel for
for(j=0; j<STREAM_ARRAY_SIZE; j++)
a[w[j]] = b[r[j]]+scalar*c[r[j]]
```

This transformation is insufficient, however, as it introduces race conditions as the indirect lookup through “w” can cause write conflicts when parallelized. To address this issue, the arrays “a”, “b”, and “c” had guard buffers added on either side thereby ensuring each thread operates within its own partitioned index space with no conflicts. This has the additional benefit of more closely mimicking the parallelization strategy of most of our codes which also use explicit partitioning. This modified variant of STREAM will be referred to as jSTREAM throughout.

```
#pragma omp parallel
{
    size_t offset=0;
    #ifdef OPENMP
    int id = omp_get_thread_num();
    offset = THREAD_BUF/2 * id * (
        STREAM_ARRAY_SIZE/numthreads) +
        THREAD_BUF);
    #endif
    for(j = offset; j < offset +
        STREAM_ARRAY_SIZE/numthreads; j++) {
        idxw = j+w[j-offset];
        idxr = j+r[j-offset];
        a[idxw] = b[idxr] + scalar * c[idxr];
    }
}
```

To mimic the sparsity of our codes using jSTREAM the read and write index arrays are constructed by sampling from the windowed memory distance histogram (described in Section III) on a per function basis.

Our second approach focuses on using trace replay techniques in which memory traces are captured via PIN which traces individual functions. A trace file in dynamorio format is captured for each function. Due to trace overheads and sizes of traces, only a single thread of execution is traced. These traces are then processed using a custom tool called `gs_patterns`

which finds loops by looking for repeating iaddr and then analyzes the memory instructions within these loops. Analysis is limited to 1024 unique memory instructions to reduce overall analysis overhead while capturing the complexity of memory accesses within a loop. For each gather/scatter identified within the loop, the top 5 (based on count of occurrence) a histogram of relative offsets is created. This approach is much higher fidelity than the function level histograms technique described earlier, at the cost of trace size and analysis time. These “reduced order” G/S traces are then saved to JSON format for use in Spatter.

Spatter was originally designed to handle compact scatter/gather patterns with limited complexity in a given temporal window of code execution. To accommodate the potentially large complexity of arbitrary index space indirection with no discernable “compact pattern” in our codes, Spatter was modified to read extremely large index arrays from JSON files. Spatter was further modified to use dynamically allocated arrays to store the gather/scatter offsets and their corresponding data arrays to perform the experiments, and increased the maximum length of patterns from 1,024 to 1,048,576 memory offsets. These changes are available in the devel branch of Spatter on Github ¹. Additionally, the routines that parse the JSON input to Spatter were modified to appropriately cut off patterns of length greater than 1,048,576. Spatter memory access patterns for select xRAGE and Flag simulations are also available on Github ².

The gather/scatter patterns identified from the trace capture Intel PIN Tool occasionally contained huge memory address offsets from the base address of the pattern due to accesses of dynamically allocated memory in the heap. Spatter replicates the gather/scatter patterns it ingests by allocating an array to simulate the address space required to fully contain the provided pattern (i.e. from 0, the base address, to the maximum offset from the base address). Consequently, patterns identified from the `gs_patterns` which contained large memory offsets could not be mapped to the simulated address space that Spatter could allocate. For example, Intel’s x86-64 processors use 48-bit virtual memory addressing, corresponding to 256 TB of address space and possibly resulting in virtual memory offsets of similar magnitude, while Spatter sets a maximum allocation size of several GB. To address this issue, a mapping routine was added to Spatter to take large memory offsets from the collected gather/scatter patterns and re-map them to a memory offset within the Spatter address space. This routine effectively shrinks the address space that the gather/scatter pattern operates within, as illustrated in Figure 4.

V. EXPERIMENTS AND RESULTS

Micro-benchmarks are often used to evaluate future node architecture designs and their potential performance implications via hardware level simulation. In this section a number of experiments are conducted to demonstrate the utility of the

¹<https://github.com/hpcgarage/spatter>

²<https://github.com/lanl/spatter>

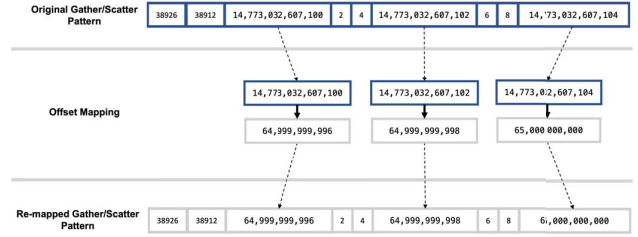


Fig. 4. Visualization of re-mapping gather/scatter patterns with huge memory offsets.

jSTREAM and modified Spatter benchmarks and the impact of memory technology and CPU architecture differences on sparse memory workloads. Analysis is conducted using both hardware simulation as well as benchmarking on real hardware. In the first experiment, gem5 was configured with Ruby to simulate a RISC-V CPU with 8, 16, and 32 cores, CPU clock rate of 3.4GHz, system clock of 2.0 GHz, L1 cache 32 kB, and L2 cache 512 kB with 8 memory channels of either LPDDR5 or HBM3 memories. jSTREAM was then configured with input arrays from representative xRAGE functions. jSTREAM was configured to use all cores on each of the 8, 16, and 32 core configurations. Performance was then measured for each configuration of hardware simulation (number of cores, memory type) for each input array. Performance of jSTREAM for each array input measured as a percentage of peak performance of HBM3 STREAM is illustrated in Figure 5. Each configuration is further broken down into 6 series, one for each memory type (HBM3 or LPDDR5) and number of CPU cores (8, 16, 32).

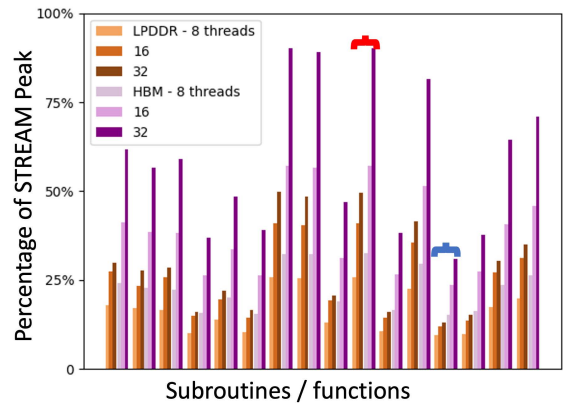


Fig. 5. Performance of xRAGE sampled functions using jSTREAM running on gem5/Ruby with HBM3 and LPDDR5 memories

The performance of functions that have less sparsity, such as cell loops across AMR leaf cells, perform well and show good scaling of bandwidth per core as illustrated by the red bracketed series in Figure 5. Near perfect scaling in bandwidth per core is achieved for HBM3 configurations with

this function's workload approaching 90% of peak stream bandwidth on HBM3. LPDDR5 configurations achieve very high relative bandwidth as well, 50% of peak bandwidth achieved with STREAM on HBM3. LPDDR5 doesn't scale as well as HBM3 with this workload, likely a result of nearing memory system peak performance at a smaller number of cores for this workload. The HBM3 configuration on the other hand requires more cores to achieve a higher percentage of peak performance. Additional cores may be required to drive full HBM3 bandwidth in this processor and memory configuration.

The performance of functions with more significant sparsity, such as face loops that indirect into neighboring cells in the mesh, show significantly lower bandwidth relative to STREAM on HBM3 as illustrated by the blue bracketed series in Figure 5. Per core bandwidth scaling for HBM3 is reasonable and peaks at nearly 30% of stream peak bandwidth. LPDDR5 configurations do not scale well with this workload with very little relative performance improvement as additional cores are used. This is likely a result of the loaded latency of LPDDR5 increasing more quickly for sparse memory operations relative to that of HBM3 with a wider interface. Further analysis is needed however to definitively identify the cause of this poor scalability.

In the second experiment, jSTREAM was used to evaluate the performance of sparse memory operations on a number of CPU based systems including Intel Cascade Lake with 6 channels of DDR4 @ 2934 MT/s, Altra Q80-30 with 8 channels of DDR4 @ 3200 MT/s, and Cavium ThunderX2 with 8 channels of LPDDR4 @ 2666 MT/s. As seen in Figure 6, performance of the sparse function exhibits low percentage of peak performance on the Altra platform achieving a maximum of 31 GB/sec at 32 threads for jSTREAM compared to over 208 GB/sec for STREAM. Performance of jSTREAM at 32 threads is 41 GB/sec compared to a maximum of 125 GB/sec with STREAM on Intel Cascade Lake, shown in Figure 7. These results are qualitatively similar to those of the gem5 simulations but differ substantially in percentage of peak STREAM performance achieved on the two systems. This is likely a result of differences in memory controllers and caches.

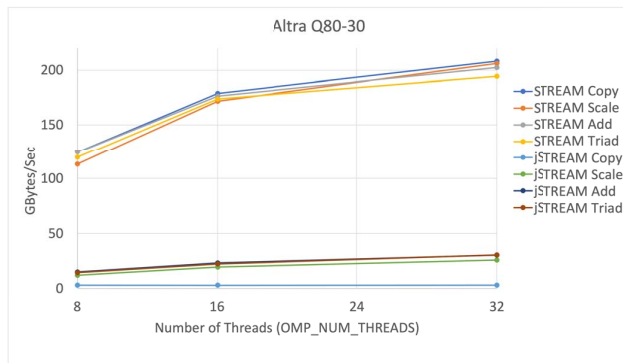


Fig. 6. Performance of STREAM benchmark and an xRAGE statistically sampled sparse function jSTREAM benchmark on Altra Q80-30

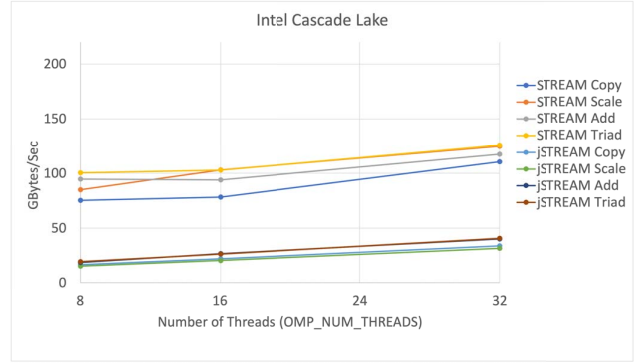


Fig. 7. Performance of STREAM benchmark and an xRAGE statistically sampled sparse function jSTREAM benchmark on Intel Cascade Lake

In the third experiment, Spatter was used to evaluate the performance of Sklyake Gold 6152 using a synthetic gather/scatter micro-benchmark. This micro-benchmark was also used to ease verification of our fine-grained tracing approach through comparison of the performance of the micro-benchmark to that of the synthetic workload generated from the benchmark and then used in Spatter. While the current implementation of `gs_patterns` is limited to serial runs future work will focus on trace capture and replay techniques to perform multi-threaded and multi-rank analyses with Spatter but may require trade-offs in the fidelity of the trace we can reasonably capture and analyze. The gather/scatter micro-benchmark consists of two kernels: gather and scatter. They each use a single level of indirection while loading or storing values of an array representing a sparse data structure.

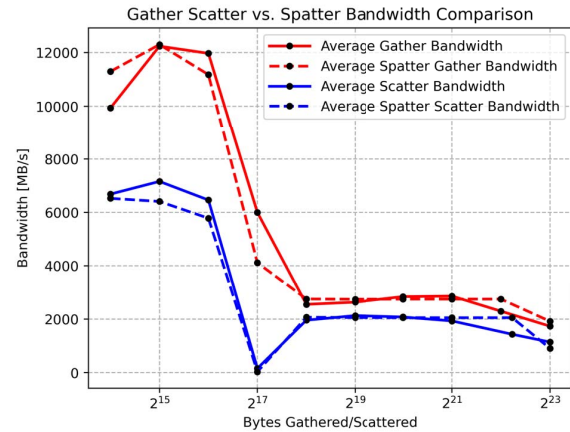


Fig. 8. Verification of Spatter Trace Replay with a Gather/Scatter Micro-Benchmark ran in Serial

The gather/scatter micro-benchmark was run with array sizes ranging from 16 kiB to 8 MiB, and each kernel gathered or scattered every item in the array. We then compared the average bandwidth over 10 runs for the gather and scatter kernels of the micro-benchmark to the trace replayed Spatter

bandwidths of the collected gather and scatter patterns, as seen in Figure 8. The average difference between the measured bandwidths of the gather kernel in the micro-benchmark and the bandwidth reported by using trace replay through Spatter was 10%. Similarly, the average difference between the micro-benchmark’s scatter kernel and the reported Spatter bandwidth was 12%.

VI. CONCLUSIONS

Many of LANL’s HPC codes are memory bandwidth bound on modern CPU architectures. Sparse memory access workloads that are non-uniform with variable stride are not well represented in common HPC benchmarks. To address this limitation, modifications to the STREAM and Spatter benchmarks enable more representative memory access patterns to be analyzed for both evaluation of existing hardware and to assess new hardware designs via simulation. As new memory technologies and new approaches to integration of these technologies are explored, there is a pressing need for more representative micro benchmarks that are tractable for hardware level simulation.

Future work is aimed at more thorough validation and consolidation of these techniques within the Spatter benchmark along with release of more robust instrumentation tools for capturing memory traces. Benchmark additions will be contributed to open source Spatter, LANL application specific patterns will continue to be released on the LANL Github site for Spatter [16].

ACKNOWLEDGMENT

The authors thank Prof. Keren Bergman from Columbia University for sharing analysis of select top 500 systems, Carola Ellinger and Daniel Holladay from LANL for additions to this, and Patrick Lavin and Jeffrey Young from GA Tech for helpful discussions on Spatter.

REFERENCES

- [1] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. The gem5 simulator. *ACM SIGARCH computer architecture news*, 39(2):1–7, 2011.
- [2] Mitsuhsa Sato, Yutaka Ishikawa, Hirofumi Tomita, Yuetsu Kodama, Tetsuya Odajima, Miwako Tsuji, Hisashi Yashiro, Masaki Aoki, Naoyuki Shida, Ikuo Miyoshi, et al. Co-design for a64fx manycore processor and” fugaku”. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2020.
- [3] June 2022 top 500. <https://www.top500.org/lists/top500/2022/06/>.
- [4] Jack J Dongarra, Piotr Luszczek, and Antoine Petit. The linpack benchmark: past, present and future. *Concurrency and Computation: practice and experience*, 15(9):803–820, 2003.
- [5] Jack Dongarra, Michael A Heroux, and Piotr Luszczek. Hpcg benchmark: a new metric for ranking high performance computing systems. *Knoxville, Tennessee*, 42, 2015.
- [6] John D McCalpin. Stream benchmark. *Link: www.cs.virginia.edu/stream/ref.html#what*, 22(7), 1995.
- [7] Leonid Oliker, Rupak Biswas, Julian Borrill, Andrew Canning, Jonathan Carter, M Jahed Djomehri, Hongzhang Shan, and David Skinner. A performance evaluation of the cray x1 for scientific applications. In *International Conference on High Performance Computing for Computational Science*, pages 51–65. Springer, 2004.
- [8] Patrick Lavin, Jeffrey Young, Jason Riedy, Richard Vuduc, Aaron Vose, and Dan Ernst. Spatter: A tool for evaluating gather/scatter performance. *arXiv preprint arXiv:1811.03743*, 2018.
- [9] Alif Ahmed and Kevin Skadron. Hopscotch: A micro-benchmark suite for memory performance evaluation. In *Proceedings of the International Symposium on Memory Systems*, pages 167–172, 2019.
- [10] Jerry S Brock. Maximizing investments in asc simulation codes at los alamos. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2018.
- [11] Donald E Burton. Lagrangian hydrodynamics in the flag code. In *Symposium on Advanced Numerical Methods for Lagrangian Hydrodynamics*, 2007.
- [12] Michael Gittings, Robert Weaver, Michael Clover, Thomas Betlach, Nelson Byrne, Robert Coker, Edward Dendy, Robert Hueckstaedt, Kim New, W Rob Oakes, et al. The rage radiation-hydrodynamic code. *Computational Science & Discovery*, 1(1):015005, 2008.
- [13] RE Alcouffe and RS Baker. Time-dependent deterministic transport on parallel architectures using partiso. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 1998.
- [14] Todd J Urbatsch and Thomas M Evans. Milagro version 2 an implicit monte carlo code for thermal radiative transfer: Capabilities, development, and usage. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2006.
- [15] Ahmad Yasin. A top-down method for performance analysis and counters architecture. In *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 35–44. IEEE, 2014.
- [16] Jered Dominguez-Trujillo, Kevin Sheridan, and Galen Shipman. Lanl/spatter. <https://github.com/lanl/spatter>.